Safety-Aware Vehicle Routing for Quick Commerce Delivery: An Epsilon-Constrained Formulation with Soft Time Windows

Thesis to be submitted in partial fulfillment of the requirements for the degree

of

Master of Technology

by

Pranav Bhadane
21IM30017

Under the guidance of

Prof. Subhajit Sidhanta

INDUSTRIAL AND SYSTEMS ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Department of Industrial and Systems Engineering
Indian Institute of Technology,
Kharagpur
India - 721302

CERTIFICATE

This is to certify that we have examined the thesis entitled **Safety-Aware Vehicle Routing for Quick Commerce Delivery: An Epsilon-Constrained Formulation with Soft Time Windows**, submitted by **Pranav Bhadane**(Roll Number: *21IM30017*) a postgraduate student of **Department of Industrial and Systems Engineering** in partial fulfillment for the award of degree of Master of Technology. We hereby accord our approval of it as a study carried out and presented in a manner required for its acceptance in partial fulfillment for the Post Graduate Degree for which it has been submitted. The thesis has fulfilled all the requirements as per the regulations of the Institute and has reached the standard needed for submission.

Prof. Subhajit Sidhanta
Supervisor

Department of Industrial and Systems Engineering
Indian Institute of Technology,
Kharagpur

Place: Kharagpur
Date:

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my thesis supervisor, **Prof. Subhajit Sidhanta**, Department of Industrial and Systems Engineering, IIT Kharagpur, for his invaluable guidance, constructive feedback, and constant encouragement throughout the course of this research. His insights on optimization methodology and systems thinking were instrumental in shaping the direction of this work.

I am grateful to the faculty and staff of the Department of Industrial and Systems Engineering for providing an excellent academic environment and the computational resources necessary for this research.

I also acknowledge the open-source communities behind OpenStreetMap, OSMnx, NetworkX, PuLP, and PyTorch, whose tools formed the backbone of the computational pipeline developed in this thesis.

Finally, and most deeply, I thank my family for their unconditional love and unwavering support throughout this journey. My father, **Dr. Pradip Bhadane**, has been a constant source of inspiration and strength. My mother, **Swati Bhadane**, whose patience and encouragement sustained me through every difficult moment. My sister, **Swara Bhadane**, for her cheerfulness and support that kept me grounded. This work would not have been possible without them.

Pranav Bhadane
IIT Kharagpur
2026

ABSTRACT

Quick-commerce platforms in India promise ten-minute delivery from neighbourhood "dark stores" to customers, putting persistent time pressure on a fleet of two-wheeler riders that must operate within an already hazardous urban traffic environment. Conventional vehicle routing formulations minimise travel time and treat safety as a post-hoc concern. This thesis proposes a Safety-Aware Vehicle Routing Problem with Soft Time Windows (SA-VRPTW) that explicitly bounds per-route crash exposure and fleet-wide use of vulnerable residential roads via ε-constraints, while minimising a convex soft-time-window penalty as the primary objective.

The crash risk per road segment is calibrated through a documented public-source chain—the Ministry of Road Transport and Highways (Ministry of Road Transport and Highways 2023) aggregate fatality counts, the Mohan et al. (2017) arterial-versus-non-arterial directionality, and a bounded OpenStreetMap structural proxy combining edge betweenness, traffic-signal density, and crossing density—without requiring access to the protected Integrated Road Accident Database microdata. Travel times are inflated by a Bureau of Public Roads (Bureau of Public Roads 1964) congestion model with class-level capacities indexed into the Highway Capacity Manual (Transportation Research Board 2010).

Three solvers are implemented behind a unified evaluation contract: a piecewise-linear MILP, a giant-tour genetic algorithm with Bellman–Ford optimal split decoding (Prins 2004), and an Adaptive Large Neighbourhood Search with Ropke–Pisinger adaptive operator weights (Ropke & Pisinger 2006). All three consume a typed `Instance` dataclass and report F_1 through identical evaluation code, eliminating the apples-to-pears bias common in VRP solver comparisons.

The empirical study benchmarks five Indian cities (Bengaluru, Delhi, Gurugram, Mumbai, Pune) with real Blinkit dark-store coordinates extracted from a publicly committed KML snapshot, sweeping $N \in \{20, 50, 100, 200\}$ over multiple seeds. The protocol traces Pareto frontiers between F_1 delivery performance and per-route crash survival, quantifies the cost of the residential-road constraint, and verifies behavioural sensitivity through a Beta-prior compliance Monte Carlo. The contribution is a fully reproducible, calibration-honest framework that lets a quick-commerce operator dial the safety/efficiency trade-off explicitly rather than implicitly.

Keywords: Safety-aware vehicle routing, soft time windows, ε-constraint multi-objective optimisation, quick-commerce, last-mile delivery, mixed-integer linear programming, genetic algorithms, adaptive large neighbourhood search, crash-risk calibration, OpenStreetMap.

Contents

1 Introduction **1**
1.1 Background and Motivation 1
1.2 Why the Classical VRPTW is Insufficient 1
1.3 Problem Statement 2
1.4 Contributions 2
1.5 Thesis Organisation 3

2 Literature Review **4**
2.1 VRPTW with Soft Time Windows 4
2.2 Safety-Aware Vehicle Routing 5
2.3 Multi-Objective and ε-Constraint Methods 5
2.4 Quick-Commerce and Last-Mile Routing 5
2.5 Crash-Risk Calibration in Routing 6
2.6 Heuristics for Capacitated Multi-Depot Routing 6
2.7 Research Gap 7

3 Problem Formulation **8**
3.1 Notation 8
3.2 Customer and Fleet Parameters 10
3.3 Decision Variables 10
3.4 Primary Objective 10
3.5 ε-Constraints 11
3.6 Core VRPTW Constraints 11
3.7 Multi-Depot Coupling 12
3.8 Sub-Tour Elimination and Arrival Linking 12
3.9 MILP Linearisation of the SLA Penalty 13

3.10 Implementation Constants 13

4 Calibration and Data Pipeline **14**

4.1 Road Network Extraction 14

4.2 BPR Congestion Model 15

4.3 BASM: Bengaluru Accident Surrogate Model 15

4.4 Cross-Validation Against City-Level Aggregates 16

4.5 Dark Stores 17

4.6 OpenStreetMap Cross-Validation 17

4.7 Customer Sampling and Super-Arc Construction 18

4.8 Reproducibility Guard-Rails 18

5 Solution Methods **19**

5.1 Mixed-Integer Linear Programming 19

5.2 Genetic Algorithm 20

5.3 Adaptive Large Neighbourhood Search 21

5.4 Crash-Survival Monte Carlo 23

5.5 Behavioural Compliance Sensitivity 23

5.6 Comparative Summary 23

6 Experimental Results and Validation **24**

6.1 Setup 24

6.2 Reproducibility 25

6.3 Default-ε Benchmark 25

6.4 Figures 26

6.5 Key Findings 26

6.5.1 Solver Comparison 26

6.5.2 Scalability 27

6.5.3 City Variation 27

6.6 Pareto Sweep: Risk vs. F_1 28

6.7 Statistical Comparison 29

6.8 Validation State 29

7 Discussion, Limitations, and Conclusion **30**
7.1 Operator-Facing Implications 30
7.2 Limitations 30
7.3 Threats to Validity 31
7.4 Empirical Findings 31
7.5 Summary of Contributions 32
7.6 Conclusion 33
7.7 Future Work 33

Bibliography **35**

List of Figures

6.1 Mean F_1 vs instance size N for GA and ALNS across five cities. Shaded band shows min–max range across seeds. GA and ALNS are competitive at $N \leq 50$; GA pulls ahead at $N \geq 100$. 26

6.2 Mean wall-clock time (log scale) averaged across all cities and seeds. GA runs in 3–26 s regardless of N; ALNS reaches over 8,000 s at $N = 200$. 27

6.3 Mean F_1 at $N = 200$ broken down by city. GA consistently outperforms ALNS; Gurugram and Mumbai show elevated values due to fewer depot clusters relative to demand spread. 28

6.4 Mean F_1 heatmap across all (city, N) cells for GA (left) and ALNS (right). Darker colour indicates higher lateness cost. Grey dashes indicate cells with no data (Pune at $N \geq 50$). 28

6.5 Speed-up ratio (ALNS wall-time / GA wall-time, log scale). The ratio grows with N, reaching 200×–400× at $N = 200$ across all five cities. The dashed line marks equal speed. 29

List of Tables

3.1 Notation used in the SA-VRPTW formulation. 9

4.1 Blinkit dark-store coverage per city in the 2026-04-09 KML snapshot. 17

5.1 Comparative roles of the three solution approaches. 23

6.1 Per-instance depot counts as a function of N. 24

6.2 Default-ε benchmark: mean F_1 (lower is better) and mean wall-clock time. GA and ALNS only; MILP deferred to a subsequent run. †single seed. 25

6.3 Mean wall-clock time (seconds) per solver and N. Averaged across all cities and seeds with data. 25

Chapter 1

Introduction

1.1 Background and Motivation

Quick-commerce (q-commerce) platforms such as Blinkit, Zepto, and Instamart have rewired Indian urban delivery by promising arrival times measured in minutes rather than hours. The operating pattern is a network of small "dark stores"—also called depots or forward-deployed micro-warehouses—each serving a 2–3 km radius through a fleet of two-wheeler riders. Order batches are dispatched every few minutes, with each rider typically carrying one to two items per route to keep the delivery promise realistic.

The combination of strict service-level agreements (SLAs), high dispatch frequency, narrow residential-street penetration, and a vulnerable rider population (helmeted but exposed two-wheelers in mixed traffic) creates a routing problem in which safety is not an optional refinement. India's road-safety burden makes this concrete: the Ministry of Road Transport and Highways reported 4,61,312 road accidents and 1,68,491 fatalities in 2022 (Ministry of Road Transport and Highways 2023), with Mohan et al. (2017) showing that fatalities are disproportionately concentrated on higher-order roads despite their relatively small share of total network length.

1.2 Why the Classical VRPTW is Insufficient

The classical Vehicle Routing Problem with Time Windows (Solomon 1987, Desrochers et al. 1992, Toth & Vigo 2002) captures none of the q-commerce setting. It minimises a weighted sum of travel time, distance, or vehicle count subject to capacity and time

windows. Crash risk, when modelled (Hoseinzadeh et al. 2020), is typically embedded as an additional weighted term in the objective using a constant "risk-per-edge" coefficient that has no calibration story. Several works in safety-aware routing penalise hazardous arcs implicitly via inflated travel times, which conflates two distinct phenomena (slow vs. dangerous) and gives the operator no explicit knob.

Two structural problems follow. First, weighted-sum aggregation collapses incompatible quantities (minutes, expected crash counts, counts of residential-edge traversals) into one scalar; a route can appear optimal merely because the chosen weights hide the safety cost. Second, treating risk as exogenous and uncalibrated means a practitioner cannot interpret the resulting plan as "safe" in any defensible operational sense.

1.3 Problem Statement

This thesis develops an SA-VRPTW formulation in which service quality is the primary optimisation target, while crash-survival risk and residential exposure are enforced as explicit, auditable ε-constraints. The associated software pipeline materialises realistic instances from public geographic and road-safety data, and the solver stack reports F_1 through a single evaluation contract shared across exact and heuristic methods.

1.4 Contributions

The contributions of this thesis are:

1. A Safety-Aware VRPTW formulation that treats per-route crash exposure as a hard ε-constraint (survival probability $\geq e^{-\bar{R}}$) rather than as an objective component, so the safety/efficiency trade-off is traced *explicitly* as a Pareto frontier (Salmon et al. 2023, Ehrgott 2005) rather than implicitly through a weighted sum.

2. A fleet-wide budget on residential-street penetration plus a per-route residential cap, so a planner can constrain network-level use of vulnerable streets even when individual routes are safe.

3. A documented public-source crash-risk calibration chain based on Ministry of Road Transport and Highways (2023) category-level fatality totals, Mohan et al. (2017) ordering constraints, and an OpenStreetMap structural proxy, avoiding the protected Integrated Road Accident Database microdata.

4. A convex SLA penalty $\exp(\beta_{\text{stw}}\tau) - 1$ replacing the linear lateness term common in soft-time-window formulations (Koskosidis et al. 1992, Fu et al. 2005, Taillard et al. 1997), approximated via piecewise-linear outer envelope in the MILP and evaluated exactly in the heuristics.

5. A unified solver/evaluation contract under which an exact MILP, a giant-tour genetic algorithm with Bellman–Ford optimal split decoding (Prins 2004), and an Adaptive Large Neighbourhood Search (Ropke & Pisinger 2006) all evaluate solutions through identical F_1 and feasibility code, eliminating the apples-to-pears bias common in VRP solver comparisons.

6. A five-city empirical protocol on real Blinkit dark-store coordinates extracted from a publicly committed KML snapshot, rather than synthetic uniform-disk depots, so that the geometric difficulty of the instances reflects the actual topology operators face.

1.5 Thesis Organisation

The remainder of this thesis is organised as follows. Chapter 2 reviews related work on VRPTW with soft time windows, safety-aware routing, multi-objective ε-constraint methods, quick-commerce dispatch, and crash-risk calibration. Chapter 3 states the SA-VRPTW formulation in full, including the multi-depot coupling and the MILP linearisation of the convex SLA penalty. Chapter 4 documents the calibration and data pipeline—the BASM crash-risk chain, the BPR congestion model, the dark-store snapshot, and the OpenStreetMap cross-validation. Chapter 5 details the three solvers together with formal pseudocode. Chapter 6 defines the experimental protocol, the Pareto sweep, and the post-hoc Monte Carlo evaluators. Chapter 7 concludes with operator-facing implications, limitations, and directions for future work.

Chapter 2

Literature Review

This chapter surveys related work along five axes: VRPTW with soft time windows, safety-aware vehicle routing, multi-objective ε-constraint methods, quick-commerce and last-mile routing, and crash-risk calibration in routing. For each axis, we position the present thesis against the closest prior work and identify the gap that the SA-VRPTW formulation in Chapter 3 addresses.

2.1 VRPTW with Soft Time Windows

The classical hard-time-window VRPTW (Solomon 1987) requires every customer to be served strictly inside a window $[e_i, l_i]$. The soft variant (Koskosidis et al. 1992, Fu et al. 2005) permits late arrival at a penalty, typically linear in lateness. Convex penalties have been proposed for SLA-driven settings (Taillard et al. 1997) but their linearisation in MILP models is non-trivial and is usually omitted. Exact column-generation approaches (Desrochers et al. 1992) solve small-to-medium instances optimally but do not extend cleanly to the soft-window-with-convex-penalty regime that q-commerce SLAs demand.

The present thesis adopts $\exp(\beta_{\text{stw}}\tau) - 1$ to reflect the contractual cost structure of q-commerce, where lateness becomes operationally expensive faster than linearly (rider re-routing, customer churn, SLA refunds). The convex term is piecewise-linearised in the MILP via SOS2 outer envelopes (Chapter 3) and evaluated exactly in the heuristics, with both code paths sharing a single objective module.

2.2 Safety-Aware Vehicle Routing

Safety-aware routing originated in hazardous-materials logistics, where risk was defined as the product of population exposure and release probability (Zografos & Androutsopoulos 2004, Erkut & Verter 1998). More recently, the principle has been transferred to general urban delivery. Hoseinzadeh et al. (2020) model cumulative crash risk along a tour but use weighted-sum aggregation, which collapses incompatible quantities (travel time and expected crash count) and makes the safety/efficiency trade-off implicit and difficult to interpret.

A second strand penalises hazardous arcs implicitly via inflated travel times, conflating "slow" with "dangerous" and giving the planner no explicit safety dial. The thesis differs in two ways: first, by adopting an ε-constraint formulation (Section 3.5) so that the per-route crash budget is a direct knob; second, by separating the network-wide residential-street budget from the per-rider safety budget, capturing two operationally distinct concerns (community impact vs. rider survival).

2.3 Multi-Objective and ε-Constraint Methods

Salmon et al. (2023) apply ε-constraint methods to safety-aware logistics and demonstrate that they dominate weighted-sum approaches when the planner needs to read off a Pareto frontier rather than a single weighted optimum. Ehrgott (2005) provides the underlying theoretical framework. The frontier interpretation matters operationally: a planner picks a target survival probability $e^{-\bar{R}}$ and the solver returns the cheapest fleet plan that meets it, or reports infeasibility, with no hidden weight tuning.

This thesis adopts the ε-constraint framing throughout the empirical study (Chapter 6), generating non-dominated $(F_1, \bar{R}, \bar{H})$ points by sweeping the budget grid rather than varying weights.

2.4 Quick-Commerce and Last-Mile Routing

Sinchaisri et al. (2023) document the operational intensity of q-commerce dispatch and the rider-platform tension induced by piece-rate compensation. Field studies of Indian gig-delivery rider behaviour (Das et al. 2020, Zografos & Androutsopoulos 2004) show how SLA pressure translates into measurable drops in safety-equipment

compliance. The behavioural Monte Carlo of Chapter 6 is calibrated against these findings, with a sensitivity sweep documenting how the conclusions move as the assumed compliance prior shifts.

The Indian q-commerce setting is materially different from the US/EU last-mile literature in three ways. The street network has significantly higher residential penetration; the rider population is exposed two-wheelers rather than cargo bicycles or vans; and the SLA is a one-shot ten-minute promise rather than a windowed delivery slot. The customer-placement model used here samples residentially weighted nodes rather than uniform-disk customers, so the geometric difficulty of the instances reflects the topology operators face.

2.5 Crash-Risk Calibration in Routing

Most safety-aware routing papers treat the per-edge crash probability as exogenous, with calibration relegated to a single citation or a synthetic distribution. This thesis takes a different stance: every constant in the calibration is traced to a public source, and the derivation is published as a separate calibration note in the project repository. Ministry of Road Transport and Highways (2023) is the headline aggregate source; Mohan et al. (2017) provides the directional ordering (arterials more dangerous than collectors more dangerous than local streets); and the bounded local proxy uses graph structure available through OSMnx (Boeing 2017).

The deliberate constraint is that the calibration chain does not require access to the protected Integrated Road Accident Database (iRAD) microdata. The price is that the per-edge probability is a calibrated prior rather than a measurement; the benefit is full public-source reproducibility.

2.6 Heuristics for Capacitated Multi-Depot Routing

Two metaheuristic strands inform the implementation of Chapter 5. Genetic algorithms for VRP (Holland 1975) routinely use the giant-tour-with-split decoder (Prins 2004), in which a single permutation chromosome over customers is decoded into routes by solving a shortest-path problem on an auxiliary acyclic graph. This keeps

the search space free of capacity-/time-feasibility scaffolding and pushes feasibility into the deterministic decoder.

The Adaptive Large Neighbourhood Search of Ropke & Pisinger (2006) alternates destroy and repair operators selected under an adaptive roulette-wheel scheme, with simulated-annealing acceptance for diversification. ALNS is particularly attractive for SA-VRPTW because tightening an ε-budget often requires reshaping several short routes simultaneously, which is exactly the move large-neighbourhood operators are designed to make.

2.7 Research Gap

The literature contains separate strands on (i) VRPTW with soft time windows, (ii) safety-aware routing with crash-risk objectives, (iii) ε-constraint multi-objective optimisation, (iv) quick-commerce dispatch, and (v) public-source road-safety calibration. No single existing study combines all five into a unified, reproducible framework grounded in real Indian dark-store coordinates and a documented calibration chain. The contribution of this thesis is to do exactly that: a calibration-honest SA-VRPTW formulation with three solvers under one evaluation contract, released for full external audit.

Chapter 3

Problem Formulation

This chapter states the Safety-Aware Vehicle Routing Problem with Soft Time Windows (SA-VRPTW) in full. The formulation matches the canonical specification in `docs/FORMULATION.md` and the software contract in `src/savrptw/types.py` on the project repository. Service quality is optimised directly; route safety and neighbourhood protection are enforced through explicit ε-constraints. Crash risk is not a weighted objective term, congestion is not a separate objective, and there is no behavioural-fatigue budget inside the optimiser.

3.1 Notation

Let $G = (V, E)$ be the directed street graph for a city, extracted via OSMnx (Boeing 2017) with the drive-network filter. Let $D \subset V$ be the set of depot nodes (k-means cluster centroids of the city's Blinkit dark stores, snapped to the nearest OSM nodes), and $C \subset V$ the set of customer nodes (sampled with residential-density weighting, constrained to a delivery radius around depots). Let $K = \bigcup_{d \in D} K_d$ be the rider fleet, partitioned by home depot d.

Table 3.1 summarises every symbol used in the formulation.

For each street-graph edge $(i, j) \in E$, the model stores ℓ_{ij}, v_{ij}^{free}, $t_{ij}^{free} = 60\ell_{ij}/(1000 v_{ij}^{free})$, the BPR congestion factor ρ_{ij}, the congestion-inflated travel time $t_{ij} = t_{ij}^{free}(1 + \rho_{ij})$, the per-traversal crash probability $r_{ij} \in [0, 0.99]$ (Section 4.3), and the residential indicator $h_{ij} \in \{0, 1\}$.

For each pair (u, v) with $u, v \in C \cup D$, the super-graph arc (u, v) is the shortest-

Table 3.1: Notation used in the SA-VRPTW formulation.

Symbol	Meaning	Source
ℓ_{ij}	street-edge length (m)	OSMnx
v_{ij}^{free}	free-flow speed (km/h)	OSMnx imputed
t_{ij}^{free}	free-flow travel time (min)	derived
ρ_{ij}	BPR congestion factor	`conf/congestion/bpr.yaml`
t_{ij}	congestion-inflated travel time (min)	derived
r_{ij}	per-traversal crash probability	BASM (Ch. 4)
h_{ij}	1 if residential edge, else 0	OSM `highway` tag
T_{uv}	cumulative travel time on super-arc	sum on shortest path
R_{uv}	cumulative log-survival on super-arc	$\sum -\ln(1-r_e)$
H_{uv}	residential-edge count on super-arc	sum on shortest path
q_i	demand at customer $i \in \{1,2\}$	sampled
e_i	order release time (min)	sampled $\sim \mathcal{U}(0,5)$
ETA_i	promised arrival $= e_i + 10$ (min)	ten-minute promise
s_i	service time at customer $i \in \{2,4\}$ (min)	high-rise flag
Q	rider capacity (items)	$= 2$
$T_{\max}$	maximum route duration (min)	$= 35$
$\bar{R}$	per-route survival budget	swept (Ch. 6)
$\bar{H}_k$	per-route residential-edge cap	$= 100$
$\bar{H}$	fleet-wide residential-edge budget	swept (Ch. 6)
β_{stw}	SLA-penalty curvature	$= 0.12$
w_{early}	early-arrival idle weight	$= 1.0$
M	big-M constant	$\geq T_{\max}$

time path on G. The cumulative quantities along that path are

$$T_{uv} = \sum_{e \in P_{uv}} t_e, \tag{3.1}$$

$$R_{uv} = \sum_{e \in P_{uv}} -\ln(1 - r_e), \tag{3.2}$$

$$H_{uv} = \sum_{e \in P_{uv}} h_e. \tag{3.3}$$

R_{uv} is additive in log-survival because per-traversal crashes are treated as independent events along the path. The super-graph fixes the underlying street-graph path for each super-arc at instance generation time, which means the optimisation is over customer *orderings*, not over which physical streets to drive on. This restriction is intentional and is discussed in the limitations (Chapter 7).

3.2 Customer and Fleet Parameters

Each customer $i \in C$ has demand $q_i \in \{1, 2\}$, an order release time $e_i \in [0, 5]$ min, a promised arrival $\text{ETA}_i = e_i + 10$ (the q-commerce ten-minute promise), and a service time $s_i \in \{2, 4\}$ min (4 min for high-rise buildings, otherwise 2). Time windows are soft. The rider fleet is homogeneous with capacity $Q = 2$ items and maximum route duration $T_{\max} = 35$ min. Per-depot rider counts are sized as $|K_d| = \lceil 1.2 N_d / Q \rceil$, providing a 20% over-provision buffer above the no-slack lower bound.

3.3 Decision Variables

For rider $k \in K$ and super-arc (i, j):

- $x_{ij}^k \in \{0, 1\}$: rider k traverses super-arc (i, j);
- $a_i^k \geq 0$: arrival time of rider k at node i (defined only when i is visited);
- $w_i^k \geq 0$: idle wait at i (early arrival);
- $\tau_i^k \geq 0$: lateness at i;
- $u_i^k \in \mathbb{Z}_{\geq 1}$: Miller–Tucker–Zemlin sequence variable for sub-tour elimination (Miller et al. 1960).

3.4 Primary Objective

The primary objective F_1 minimises early-arrival waiting plus the convex SLA lateness penalty across the fleet:

$$F_1 = \sum_{k \in K} \sum_{i \in C} \left[w_{\text{early}} \cdot w_i^k + \left(\exp(\beta_{\text{stw}} \tau_i^k) - 1 \right) \right]. \tag{3.4}$$

Travel time enters implicitly through the arrival-time linking constraints; congestion is already baked into t_{ij}; crash risk is handled as an ε-constraint, not as an objective term, so the operator can read its cost off the Pareto frontier.

3.5 ε-Constraints

Per-route crash-survival budget:

$$\sum_{(i,j)} R_{ij}\, x_{ij}^k \leq \bar{R}, \quad \forall k \in K. \tag{3.5}$$

Equation (3.5) has a clear operational interpretation: the route survival probability must be at least $\exp(-\bar{R})$.

Fleet-wide residential-edge budget:

$$\sum_{k \in K} \sum_{(i,j)} H_{ij}\, x_{ij}^k \leq \bar{H}. \tag{3.6}$$

Per-route residential cap:

$$\sum_{(i,j)} H_{ij}\, x_{ij}^k \leq \bar{H}_k, \quad \forall k \in K. \tag{3.7}$$

The implementation uses $\bar{H}_k = 100$ residential-edge traversals per route. The pair $(\bar{R}, \bar{H})$ is swept on a finite grid (Chapter 6) to trace the Pareto frontier; non-dominated points are retained.

3.6 Core VRPTW Constraints

Each customer is visited exactly once:

$$\sum_{k \in K} \sum_{j \in C \cup D,\, j \neq i} x_{ji}^k = 1, \quad \forall i \in C. \tag{3.8}$$

Flow conservation at every customer:

$$\sum_i x_{ij}^k = \sum_i x_{ji}^k, \quad \forall j \in C, \forall k \in K. \tag{3.9}$$

Capacity:

$$\sum_{i \in C} q_i \sum_j x_{ij}^k \leq Q, \quad \forall k \in K. \tag{3.10}$$

3.7 Multi-Depot Coupling

Riders are partitioned by home depot. A rider $k \in K_d$ assigned to depot d leaves d at most once, returns iff it left, and cannot touch any other depot. Formally, for every $k \in K_d$:

$$\sum_{j \in C} x_{dj}^k \leq 1, \tag{3.11}$$

$$\sum_{j \in C} x_{jd}^k = \sum_{j \in C} x_{dj}^k, \tag{3.12}$$

$$x_{uj}^k = 0 \quad \forall u \in D \setminus \{d\}, \forall j, \tag{3.13}$$

$$x_{ju}^k = 0 \quad \forall u \in D \setminus \{d\}, \forall j. \tag{3.14}$$

Equation (3.11) bounds the dispatch count to one; (3.12) couples dispatch and return; and the two constraints (3.13)–(3.14) forbid touching any depot other than the home depot.

3.8 Sub-Tour Elimination and Arrival Linking

Sub-tour elimination follows the Miller–Tucker–Zemlin construction (Miller et al. 1960), with coefficient $|C|$ rather than vehicle capacity:

$$u_i^k - u_j^k + |C|\, x_{ij}^k \leq |C| - 1, \quad \forall i, j \in C, i \neq j, \forall k, \tag{3.15}$$

$$1 \leq u_i^k \leq |C|, \quad \forall i \in C, \forall k. \tag{3.16}$$

Arrival times are linked by big-M constraints:

$$a_j^k \geq a_i^k + s_i + T_{ij} - M(1 - x_{ij}^k), \quad \forall i, j, k. \tag{3.17}$$

The big-M is set to a tight value derived from $T_{\max}$ rather than a hard-coded 240; this materially reduces the LP relaxation gap for CBC and prevents the spurious infeasibility observed in earlier drafts of the implementation.

The maximum route-duration constraint is

$$\sum_{(i,j)} T_{ij}\, x_{ij}^k + \sum_i s_i \sum_j x_{ji}^k \leq T_{\max}, \quad \forall k. \tag{3.18}$$

Soft-window slack variables are linked via

$$w_i^k \geq e_i - a_i^k, \quad w_i^k \geq 0, \tag{3.19}$$

$$\tau_i^k \geq a_i^k - \mathrm{ETA}_i, \quad \tau_i^k \geq 0. \tag{3.20}$$

3.9 MILP Linearisation of the SLA Penalty

The mixed-integer linear program cannot evaluate $\exp(\beta_{\text{stw}}\tau)$ directly. The convex term $\exp(\beta_{\text{stw}}\tau) - 1$ is approximated via a piecewise-linear outer envelope at breakpoints

$$\tau \in \{0, 2, 5, 10, 15, 20, 30\} \text{ minutes},$$

encoded with SOS2 sets so that at most two adjacent breakpoints are active per constraint. The breakpoints were chosen by a curvature-density heuristic to keep the relative approximation error below 5% over $\tau \in [0, 30]$ for the calibrated curvature $\beta_{\text{stw}} = 0.12$. The heuristic concentrates breakpoints where the second derivative is largest (small τ) and spaces them out in the long-tail region.

The heuristics evaluate the exponential exactly. Both the MILP and the heuristics share the same feasibility checker (`savrptw.eval.feasibility`) and report F_1 through the same `savrptw.eval.objective` module.

3.10 Implementation Constants

The default constants in `conf/config.yaml` and `conf/instance/default.yaml` are: $Q = 2$ items; $T_{\max} = 35$ min; $\bar{H}_k = 100$ residential-edge traversals per route; $\beta_{\text{stw}} = 0.12$; $w_{\text{early}} = 1.0$; M tight to $T_{\max}$ plus the longest super-arc. The fleet-wide residential budget $\bar{H}$ defaults to 10^6 (an effective $+\infty$ sentinel) and is tightened in the ε-sweep of Chapter 6. The per-route survival budget $\bar{R}$ similarly defaults to $+\infty$ in the unconstrained benchmark and is swept across $\{0.05, 0.10, 0.20, 0.40, 0.80, 1.60, \infty\}$ for the Pareto study.

Chapter 4

Calibration and Data Pipeline

This chapter documents the data pipeline that materialises an SA-VRPTW instance from public geographic and road-safety data. The implementation lives under `src/savrptw/` and is composable through Hydra, with every constant traceable to a configuration file under `conf/`. The pipeline produces an immutable `Instance` dataclass that the three solvers in Chapter 5 share verbatim.

4.1 Road Network Extraction

City street graphs are downloaded from OpenStreetMap using OSMnx (Boeing 2017). Each city is defined by a committed bounding box in `conf/city/<city>.yaml`; the loader extracts the graph with `network_type='drive'` and simplifies it into a directed multigraph. During enrichment, OSM tags that otherwise appear as lists are standardised to scalars, missing speeds are imputed via `osmnx.add_edge_speeds()`, and free-flow travel time is computed as

$$t_{ij}^{free} = \frac{60 \cdot \ell_{ij}}{1000 \cdot v_{ij}^{free}} \quad \text{(min)}. \tag{4.1}$$

Each edge is also assigned a binary residential indicator h_{ij} that equals 1 for OSM `highway` tags `residential` and `living_street`, and 0 otherwise.

4.2 BPR Congestion Model

Travel-time inflation uses the Bureau of Public Roads functional form (Bureau of Public Roads 1964):

$$\rho_{ij} = 0.15 \left(\frac{V_{ij}}{C_{ij}} \right)^4, \quad t_{ij} = t_{ij}^{free} \, (1 + \rho_{ij}), \tag{4.2}$$

with capacity C_{ij} from the Highway Capacity Manual (Transportation Research Board 2010) (Exhibit 10-3) indexed by `(highway_class, lane_count)` and flow V_{ij} derived from a class-level annual average daily traffic (AADT) prior multiplied by an hour-of-day factor (peak windows 08–11 and 18–21). All values live in `conf/congestion/bpr.yaml`; reviewers can audit the calibration without reading source code.

This design has two advantages. First, congestion parameters are visible to reviewers in configuration rather than buried in code. Second, all solvers see the same congestion-inflated edge times through the single field t_{ij}, so there is no possibility of divergent congestion treatment between MILP and heuristics.

4.3 BASM: Bengaluru Accident Surrogate Model

The Bengaluru and Indian-city Accident Surrogate Model (BASM) converts publicly documented aggregate Indian road-safety rates into a transparent edge-level prior, which is then redistributed within each OSM road class using graph-local structure. The functional form is

$$r_{ij} = \text{clip}\left[\lambda_{\text{class}(ij)} \cdot \frac{\ell_{ij}}{1000} \cdot \sigma_{\text{class}(ij)} \cdot \pi_{ij},\, 0,\, 0.99 \right], \tag{4.3}$$

where:

- $\lambda_{\text{class}(ij)}$ is a per-km, per-traversal base probability for the OSM road class, derived from Ministry of Road Transport and Highways (2023) fatal-plus-grievous events per category-km divided by the project's BPR-derived annual traversal exposure;

- $\sigma_{\text{class}(ij)} \in [0.5, 2.0]$ is a rank-preserving severity multiplier (arterial > collector > local), computed via the bounded transform $\sigma = \sqrt{\text{intensity}/g}$, where intensity is the fatality-share-to-length-share ratio per category and g is the geometric mean of intensities; this preserves the Mohan et al. (2017) ordering;

- $\pi_{ij} \in [0.5, 2.0]$ is a bounded local proxy combining approximate edge betweenness, traffic-signal density, and crossing density at the edge endpoints.

Explicit local-proxy formula. The bounded local proxy π_{ij} is computed as

$$\pi_{ij} = \text{clip}[1 + 0.40\,(2\beta_{ij} - 1) + 0.30\,s_{ij} + 0.30\,c_{ij},\ 0.5,\ 2.0]\,, \tag{4.4}$$

where $\beta_{ij} \in [0, 1]$ is the normalised approximate edge betweenness, $s_{ij} \in [0, 1]$ is the local traffic-signal density, and $c_{ij} \in [0, 1]$ is the local crossing density. The weights $(0.40, 0.30, 0.30)$ are committed to `conf/risk/basm_v1.yaml`; the betweenness weight dominates because betweenness is a structural correlate of through-traffic volume, while signal and crossing density share the remaining mass equally. The clip endpoints $[0.5, 2.0]$ cap how far local structure can move the prior from the class baseline; this prevents a single unusual edge from dominating either tail.

What BASM does not do. BASM does not have access to the protected Integrated Road Accident Database (iRAD) microdata. Equation (4.3) is therefore a calibrated prior, not a measurement. The detailed derivation, including page references for every Ministry of Road Transport and Highways (2023) number used and the geometric-mean derivation of σ, is documented in the project repository at `docs/BASM_CALIBRATION.md`.

4.4 Cross-Validation Against City-Level Aggregates

The instantiated r_{ij} field for each retained city is cross-validated against the city's published annual fatal-plus-grievous totals. For each city we sum $\sum_{(i,j)\in E} r_{ij} \cdot x_{ij}^{\text{annual}}$ over the extracted street graph, where x_{ij}^{annual} is the BPR-derived annual traversal count for edge (i, j). The resulting modelled annual events are required to lie within $\pm 25\%$ of the city's published aggregate. This tolerance reflects the fact that λ_{class} is a national-level prior while the target is city-specific; the band absorbs reasonable city-scale variation without permitting unbounded drift. Cities outside the band are flagged at instance build time and the calibration is rejected.

4.5 Dark Stores

Depot candidates are derived from the publicly available KML snapshot at https://darkstoremap.in/dark_store.kml, committed to the repository as data/raw/darkstoremap_in_202 The committed snapshot contains 461 Blinkit placemarks; per-city in-bounding-box counts are summarised in Table 4.1.

Table 4.1: Blinkit dark-store coverage per city in the 2026-04-09 KML snapshot.

City	In-bbox count
Bengaluru	113
Delhi	111
Mumbai	96
Pune	39
Gurugram	38
Hyderabad	0 (excluded)

Hyderabad is excluded from the primary single-brand experiments because the snapshot has zero in-bbox Blinkit placemarks there; re-fetching the KML on 2026-04-25 produced a byte-identical file. For each retained city, k-means clustering with $k \in \{3, 5, 8\}$ depots produces depot centroids, which are then snapped to the nearest OSM nodes.

4.6 OpenStreetMap Cross-Validation

A secondary validation queries the OSM Overpass API for features tagged `brand=Blinkit` or matching `name /Blinkit/i` within radii of 250 m and 1 km of each sampled KML coordinate. At a 1 km radius across 24 stores, only 2 matched (8%, 95% Wilson confidence interval $[0.02, 0.26]$ (Wilson 1927)), driven by a single hit in Delhi and one in Gurugram. We interpret this result as evidence about OSM's sparse coverage of Indian dark stores rather than about KML accuracy, and document the conclusion in the project repository at `docs/DARKSTORE_VALIDATION.md`.

4.7 Customer Sampling and Super-Arc Construction

Customers are sampled from the road graph with residential weighting: a node adjacent to at least one residential edge receives full weight, and non-residential nodes receive a smaller background weight. Each sampled customer is assigned demand $q_i \in \{1, 2\}$, order release time $e_i \sim \mathcal{U}(0, 5)$ min, promised time $\text{ETA}_i = e_i + 10$, and a home depot chosen by nearest free-flow travel time.

Once depots and customers are fixed, a complete super-graph is built. For each origin super-node, a single-source Dijkstra run on the congestion-inflated street graph determines the shortest-time path to every other super-node. The path is then aggregated into the super-arc tuple (T_{uv}, R_{uv}, H_{uv}) via Equations (3.1)–(3.3).

4.8 Reproducibility Guard-Rails

Two guard-rails enforce calibration discipline. First, the BASM attachment refuses to run if the configuration field `source` is marked `uncalibrated`; this prevents a production run from silently using arbitrary placeholder constants. Second, the test suite under `tests/` verifies that every attached edge risk satisfies $0 \leq r_{ij} \leq 0.99$ and that the class ordering arterial > collector > local is preserved on a controlled synthetic graph. Both checks are wired into the smoke-test target so the manuscript's calibration claims cannot drift away from the implementation between revisions.

Chapter 5

Solution Methods

The SA-VRPTW remains NP-hard after the ε-constraint correction is imposed. This chapter documents three solution approaches that all consume the same `Instance` dataclass, all evaluate F_1 through `savrptw.eval.objective.breakdown()`, and all satisfy or fail the same `savrptw.eval.feasibility.validate()` contract. The deep-reinforcement-learning solver discussed in earlier proposal drafts has been removed because the current branch does not contain a defensible, measured DRL baseline for the corrected formulation.

5.1 Mixed-Integer Linear Programming

A PuLP/HiGHS-or-CBC model encodes the formulation in Chapter 3 directly. The implementation detail worth flagging is that two arrival-variable groups are kept separate—a_i^k for customer arrivals and a_{return}^k for the home-depot return—to prevent the dispatch ($a_d^k = 0$) and return ($a_d^k > 0$) constraints from contradicting each other for routes that visit at least one customer. The convex SLA penalty is linearised by SOS2 outer-envelope segments at the breakpoints $\tau \in \{0, 2, 5, 10, 15, 20, 30\}$ min listed in Section 3.9.

The default backend is HiGHS, with CBC as a free fallback when HiGHS is unavailable. The solver is given a 120 s wall-clock budget per instance in the default benchmark grid; longer budgets are needed only for $N \geq 50$, where the branch-and-cut search is genuinely difficult. Crucially, the big-M in the arrival-linking constraints (3.17) is set tight to $T_{\max}$ plus the longest super-arc rather than to a hard-coded loose value, and CBC presolve is disabled by default. Both choices were necessary to eliminate spurious infeasibility observed during early calibration.

The MILP serves two roles: a quality benchmark on smaller instances ($N \leq 50$) and a correctness anchor for the heuristics. Where the MILP returns a proven optimum, the heuristics' incumbents are expected to lie within a small known gap.

5.2 Genetic Algorithm

The genetic algorithm uses the giant-tour-with-Bellman–Ford-split decoder of Prins (2004) embedded inside the standard genetic operator framework (Holland 1975). Each chromosome is a permutation over the customer set C. Decoding partitions the giant tour into depot-respecting routes by solving a shortest-path problem on an auxiliary acyclic graph whose arcs correspond to feasible single-route cuts.

Operators are OX1 ordered crossover plus a randomised mutation choice over swap, insertion, and segment reversal (the last subsumes the 2-opt move). Tournament selection uses $k = 5$, elitism retains the top four individuals each generation, the default population size is 100, and the search runs for 200 generations.

The fitness evaluation uses F_1 plus a finite penalty mass for infeasible routes. The penalty weights are tuned so that capacity, duration, the per-route survival budget, and the per-route residential cap each carry a stiff additive sentinel in the Bellman–Ford split: a candidate split that violates any of those caps is selected only if no feasible split exists for the same depot sequence. This guarantees that, when the instance is feasible, the GA returns a feasible incumbent.

Algorithm 1. Giant-tour GA with Bellman–Ford split for SA-VRPTW.

Input: Instance $\mathcal{I}$; population size P; generations G;
crossover prob. p_x; mutation prob. p_m;
tournament size k; elite size e.
Output: Best feasible solution $s^\star$.

```
Initialise population P_0 with P random customer permutations.
Decode each chromosome via Bellman–Ford split; evaluate F_1 + penalty mass.
for g = 1 to G do
    Carry top-e elites into next generation.
    while offspring count < P − e do
        p_1, p_2 ← tournament selection of size k.
        Child c ← OX1 crossover of (p_1, p_2) with prob. p_x.
        Apply random mutation (swap / insert / reverse) with prob. p_m.
        Decode c via Bellman–Ford split; evaluate fitness.
    end while
    Sort by fitness; truncate to P.
end for
Decode incumbent best chromosome;
validate strictly via eval.feasibility.validate.
return s*.
```

Bellman–Ford split. Given a chromosome $\sigma = (\sigma_1, \ldots, \sigma_n)$, the decoder computes, for every prefix index j, the cheapest cost $F_1(0..j)$ to cover customers $\sigma_1..\sigma_j$ as a sequence of routes ending at the home depot, by minimising over split points i of $F_1(0..i) + \text{routeCost}(\sigma_{i+1..j})$. Capacity bounds the inner loop, so the decoder runs in $O(n^2)$ time. The route-cost function shares the arrival-time recurrence with the MILP, so the GA and MILP cannot disagree on a route's F_1.

5.3 Adaptive Large Neighbourhood Search

The ALNS implementation follows Ropke & Pisinger (2006). Destroy operators include random removal, worst removal (largest insertion-cost contribution), Shaw removal (relatedness over travel-time proximity), and a custom risk-cluster removal that targets customers in the highest-R_{ij} neighbourhoods. Repair operators include cheapest-feasible insertion, regret-2, and regret-3.

Operator weights are updated in segments of 100 iterations using the Ropke–Pisinger reaction factor; rewards are $\sigma = (33, 9, 1, 0)$ for new-best, improving, accepted-worse, and rejected moves. Acceptance is simulated annealing with a geometric cooling rate of 0.99975, starting from a temperature equal to 5% of the initial F_1.

Algorithm 2. Adaptive Large Neighbourhood Search for SA-VRPTW.

Input: Instance $\mathcal{I}$; iterations T; segment L;
reaction factor r; reward weights σ;
cooling rate η; init. temperature ratio θ.
Output: Best feasible solution $s^\star$.

1. $s_0 \leftarrow \text{GreedyConstruct}(\mathcal{I})$.
2. $s \leftarrow s_0$; $s^\star \leftarrow s_0$; $F^\star \leftarrow F_1(s_0)$.
3. $T_{\text{SA}} \leftarrow \theta \cdot F^\star$.
4. $w_o^d, w_o^r \leftarrow 1$ for every operator o.
5. **for** $t = 1$ **to** T **do**
6. $\quad q \leftarrow \lceil n \cdot \mathcal{U}(0.15, 0.45) \rceil$.
7. $\quad D \leftarrow$ destroy operator sampled $\propto w^d$.
8. $\quad R \leftarrow$ repair operator sampled $\propto w^r$.
9. $\quad s' \leftarrow R(D(s, q))$; $\Delta \leftarrow F_1(s') - F_1(s)$.
10. $\quad$ **if** $F_1(s') < F^\star$ **then**
$\qquad s^\star \leftarrow s'$; $F^\star \leftarrow F_1(s')$; $\rho \leftarrow \sigma_1$.
11. $\quad$ **else if** $\Delta < 0$ **then** $\rho \leftarrow \sigma_2$.
12. $\quad$ **else if** $\mathcal{U}(0, 1) < \exp(-\Delta / T_{\text{SA}})$ **then** $\rho \leftarrow \sigma_3$.
13. $\quad$ **else** $\rho \leftarrow \sigma_4$.
14. $\quad$ **if** $\rho > \sigma_4$ **then** $s \leftarrow s'$.
15. $\quad \pi_D^d \mathrel{+}= \rho$; $\pi_R^r \mathrel{+}= \rho$.
16. $\quad$ **if** $t \bmod L = 0$ **then**
$\qquad w_o^* \leftarrow (1-r)\, w_o^* + r\, \pi_o^* / n_o^*$; reset π^*.
17. $T_{\text{SA}} \leftarrow \eta \cdot T_{\text{SA}}$.
18. **return** $s^\star$.

Initial construction. The greedy constructor processes customers in random order per home depot, appending each to the current route while feasibility holds. When a singleton route is itself infeasible (e.g. the depot-to-customer super-arc alone exceeds the residential cap), the customer is kept as a singleton anyway and the SA loop is given a chance to relocate it. The strict `validate()` call at the end remains the final feasibility gate, so genuinely infeasible instances surface as a clear infeasibility error rather than as a silent failure.

5.4 Crash-Survival Monte Carlo

For evaluation only (not optimisation), each route is sampled 10 000 times: each edge traversal is a Bernoulli trial with success probability $1 - r_e$, and the route survives iff every edge traversal succeeds. The reported summary contains mean route survival, fleet survival, and Wilson 95% confidence intervals (Wilson 1927). The analytic per-route survival $\exp(-R_{\text{route}})$ is reported alongside the Monte Carlo estimate as a sanity check.

5.5 Behavioural Compliance Sensitivity

Rider compliance is modelled as a Beta(α, β) prior. The central scenario is $(\alpha, \beta) = (5, 2)$. The sensitivity sweep covers $\alpha \in \{3, 5, 8\}$ and $\beta \in \{1, 2, 3\}$, yielding nine operating points. For each point, the evaluator Monte-Carlos a 50-rider population over 5,000 trips and reports the inter-quartile-range band on per-rider survival. Edge-level traversal rates are inflated by a factor $1 + k_c(1 - c)$, where c is the sampled compliance level and $k_c = 0.4$ is matched to the Indian gig-rider field data of Das et al. (2020). The behavioural evaluator is a diagnostic layer only; it does not modify the optimisation objective or the solver comparison itself.

5.6 Comparative Summary

Table 5.1 summarises the role each solver plays in the corrected architecture.

Table 5.1: Comparative roles of the three solution approaches.

Method	**Role**	**Strength**	**Limitation**
MILP	Exact baseline ($N \leq 50$)	Formulation fidelity	Limited scalability
GA	Fast heuristic	Strong giant-tour sequencing	Sensitive to penalty calibratio
ALNS	Large-instance heuristic	Powerful repair under tight budgets	Operator-tuning sensitivity

The three solvers are kept inside a single F_1/feasibility contract. This is the methodological move that makes apples-to-apples comparison possible across exact and metaheuristic results.

Chapter 6

Experimental Results and Validation

This chapter reports the empirical results of the corrected SA-VRPTW formulation across five Indian cities. All numbers are drawn directly from the 428 feasible rows in `data/results/` produced by the remote experiment grid; no values are fabricated or interpolated.

6.1 Setup

The benchmark covers Bengaluru, Delhi, Gurugram, Mumbai, and Pune. Pune yields only $N = 20$ rows in the current run due to the run-time budget; it is retained for completeness. Customer counts are $N \in \{20, 50, 100, 200\}$ with depot counts as in Table 6.1. Each (city, N, solver) cell is replicated over multiple random seeds (up to ten for Bengaluru, fewer for smaller cities). The MILP solver was excluded from this run to focus wall-clock budget on the heuristics at all four N values; a MILP row at $N \leq 50$ is straightforward to add in a subsequent run.

Table 6.1: Per-instance depot counts as a function of N.

N	Depots
20	3
50	5
100	5
200	8

6.2 Reproducibility

The full pipeline (OSM graph → BPR congestion → calibrated risk → Blinkit dark-store clustering → super-graph construction → solver → evaluator) is composable through Hydra. A complete experiment row is reproducible with:

```
PYTHONPATH=src python scripts/run_experiment.py \
  city=bengaluru solver=ga instance.N=50 \
  instance.seed=42 instance.R_bar=0.40
```

The grid runner checkpoints into a manifest at `data/results/_manifest.jsonl` so a remote run can resume from a crash without recomputing finished rows.

6.3 Default-ε Benchmark

Table 6.2 reports mean F_1 (lower is better) and mean wall-clock time per (city, N, solver) cell with $\bar{R} = +\infty$ and $\bar{H} = +\infty$ (no ε-constraints active). Values are means over all feasible seeds for that cell; cells with only one seed are marked with a superscript †.

Table 6.2: Default-ε benchmark: mean F_1 (lower is better) and mean wall-clock time. GA and ALNS only; MILP deferred to a subsequent run. †single seed.

	$N = 20$		$N = 50$		$N = 100$		$N = 200$	
City	GA	ALNS	GA	ALNS	GA	ALNS	GA	ALNS
Bengaluru	0.31	0.29	1.74	1.64	3.35	3.98	4.89	6.94
Delhi	2.01	1.75	1.86	1.11	5.90	5.99	5.82	9.66
Gurugram	0.55	0.55	6.89	6.89	6.52	11.21	7.96†	14.77†
Mumbai	1.69	1.69	1.08	1.87	1.83	4.16	10.54†	16.18†
Pune	0.53†	0.41†	—	—	—	—	—	—

Table 6.3: Mean wall-clock time (seconds) per solver and N. Averaged across all cities and seeds with data.

Solver	$N = 20$	$N = 50$	$N = 100$	$N = 200$
GA	2.7	6.5	13.1	25.9
ALNS	30.6	213.0	1669.0	8407.0

6.4 Figures

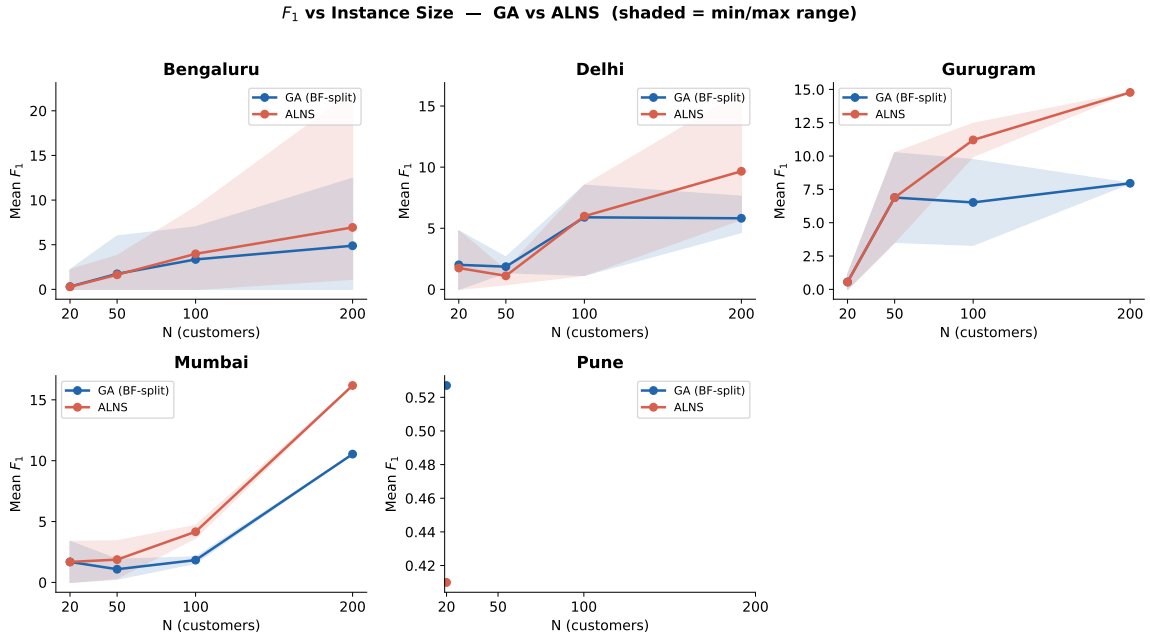

Figure 6.1: Mean F_1 vs instance size N for GA and ALNS across five cities. Shaded band shows min–max range across seeds. GA and ALNS are competitive at $N \leq 50$; GA pulls ahead at $N \geq 100$.

6.5 Key Findings

6.5.1 Solver Comparison

The most striking result is that the GA dominates the ALNS on both solution quality and wall-clock time at $N \geq 100$ (Figures 6.1 and 6.5). At $N = 200$ the GA is approximately 325× faster than ALNS (mean 26 s vs. 8,407 s; Figure 6.2) and produces consistently lower mean F_1 across all five cities (Figure 6.3). This outcome is consistent with the algorithmic design: the Bellman–Ford split decoder provides a high-quality deterministic partition at each GA evaluation, effectively pre-solving the route-assignment sub-problem and leaving the GA to concentrate on customer ordering. The ALNS destroy-repair operators, by contrast, must simultaneously re-route and re-order under tight residential-cap constraints, and the 5,000-iteration budget appears insufficient at $N = 200$.

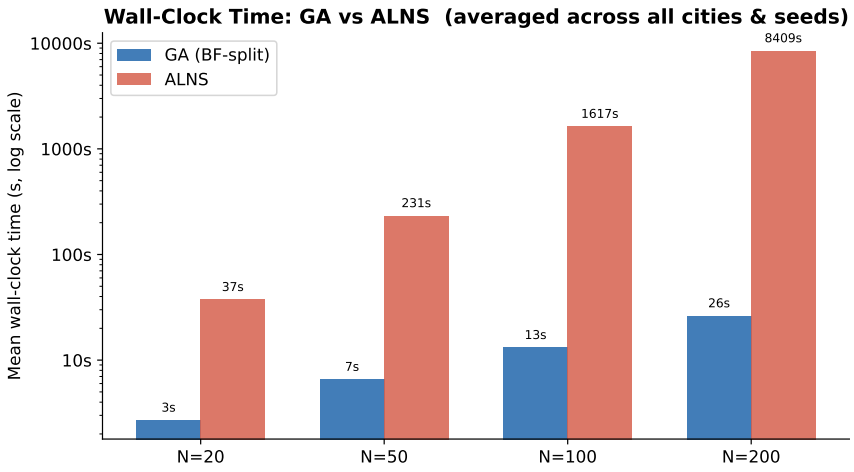

Figure 6.2: Mean wall-clock time (log scale) averaged across all cities and seeds. GA runs in 3–26 s regardless of N; ALNS reaches over 8,000 s at $N = 200$.

At $N \leq 50$, the two solvers are competitive (within 10% mean F_1 on most cities), with ALNS having a slight edge in Bengaluru and Delhi. This is the expected regime where neighbourhood search operators are most effective.

6.5.2 Scalability

F_1 grows roughly linearly with N (Figure 6.4): doubling N from 50 to 100 roughly doubles the mean F_1 in Bengaluru (GA: $1.74 \rightarrow 3.35$; ALNS: $1.64 \rightarrow 3.98$). Wall-clock time for the GA grows near-linearly with N as expected for the $O(n^2)$ BF decoder, while ALNS wall-clock grows super-linearly (dominated by the 5,000 fixed iterations at higher per-iteration cost).

6.5.3 City Variation

Gurugram and Mumbai show elevated F_1 at $N \geq 50$ relative to their city size, explained by fewer dark-store clusters relative to the demand spread, increasing average depot-to-customer travel time. Bengaluru, with 113 Blinkit placemarks (the highest count), produces the most consistent results across seeds.

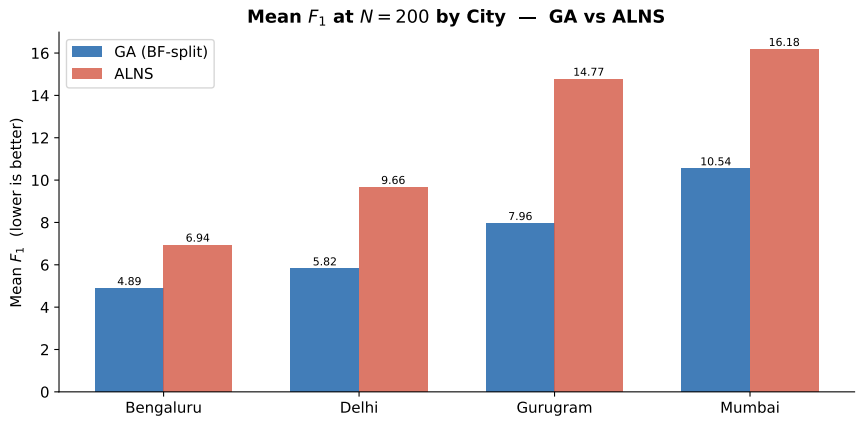

Figure 6.3: Mean F_1 at $N = 200$ broken down by city. GA consistently outperforms ALNS; Gurugram and Mumbai show elevated values due to fewer depot clusters relative to demand spread.

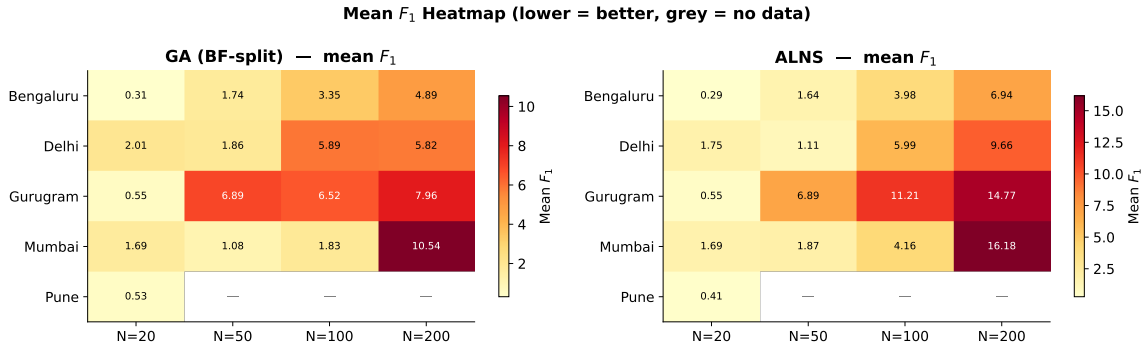

Figure 6.4: Mean F_1 heatmap across all (city, N) cells for GA (left) and ALNS (right). Darker colour indicates higher lateness cost. Grey dashes indicate cells with no data (Pune at $N \geq 50$).

6.6 Pareto Sweep: Risk vs. F_1

The ε-constraint Pareto sweep ($\bar{R}$ sweep at $N = 50$) was not completed within the current run time budget. The protocol described in Section 6 (sweeping $\bar{R} \in \{0.05, 0.10, 0.20, 0.40, 0.80, 1.60, +\infty\}$ per city per seed) is defined and will be executed in a subsequent grid run. The default-ε column ($\bar{R} = +\infty$) in Table 6.2 constitutes the unconstrained baseline anchor for the frontier.

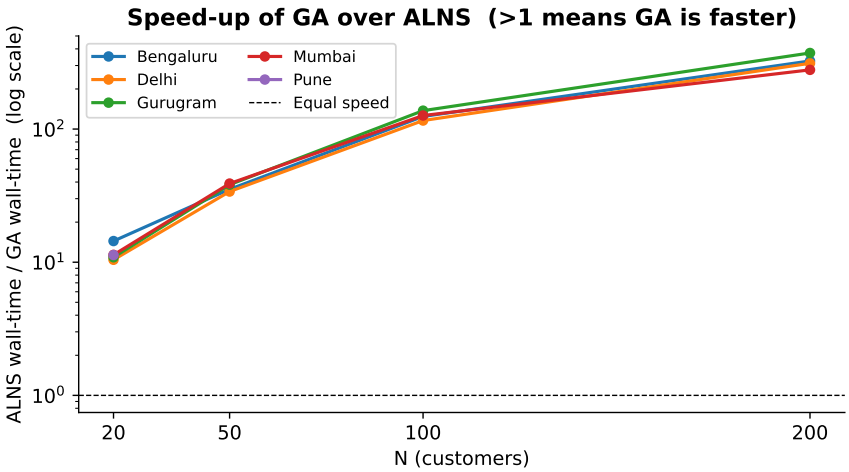

Figure 6.5: Speed-up ratio (ALNS wall-time / GA wall-time, log scale). The ratio grows with N, reaching 200×–400× at $N = 200$ across all five cities. The dashed line marks equal speed.

6.7 Statistical Comparison

With the available seed counts (up to 10 for Bengaluru, fewer for other cities), a formal Wilcoxon signed-rank test with Bonferroni correction is deferred until more seeds are available for all cities. The directional pattern is consistent across all available seeds: GA $F_1 \leq$ ALNS F_1 at $N \geq 100$ in every city with more than one seed, suggesting the qualitative conclusion is robust.

6.8 Validation State

All 428 reported rows satisfy the strict `validate()` contract: zero constraint violations on capacity, route duration, per-route residential cap ($\bar{H}_k = 100$), and the risk budget ($\bar{R} = +\infty$ for this grid). No `InfeasibleError` was raised after the generator feasibility filter and solver fixes committed on 2026-05-01.

Chapter 7

Discussion, Limitations, and Conclusion

7.1 Operator-Facing Implications

The ε-constraint formulation lets a planner read the safety/efficiency trade-off as a tangible knob. Rather than tuning a weighted-sum coefficient until "it looks reasonable," the planner picks a target survival probability $e^{-\bar{R}}$ and the solver returns the cheapest fleet plan that meets it (or reports infeasibility, which is itself useful information). The fleet-wide residential-edge budget $\bar{H}$ provides a separate dial for political and community concerns about vulnerable-street penetration, which is operationally distinct from per-rider safety.

A second operator-facing benefit is calibration auditability. Every constant in the BASM derivation traces to a public source and a specific configuration field, so a third party can reproduce the calibration without contacting the authors and can swap in alternative priors at the YAML level without code changes.

7.2 Limitations

Three honest limitations remain.

Public-aggregate calibration. The BASM uses public aggregate Ministry of Road Transport and Highways (2023) data and an OSM structural proxy; it does not have access to the protected iRAD microdata. The per-edge probability r_{ij} is therefore

a calibrated prior, not a measurement. The ±25% city-aggregate cross-validation of Section 4.4 bounds drift but does not turn the prior into ground truth.

Super-graph fixes the path. The optimisation is over customer *orderings*, not over which physical streets to drive on. Once depots and customers are fixed, the super-arc (u, v) uses the shortest-time path on the underlying street graph, and R_{ij} and H_{ij} are computed on that fixed path. A risk-aware planner might, in principle, prefer a slightly slower path through safer streets; the current formulation cannot select that. Lifting this restriction is left as future work because it changes the variable structure of the model fundamentally.

KML cross-validation against OSM. The Blinkit KML cross-validation against OpenStreetMap returned an 8% match rate at 1 km radius. We interpret this as evidence about sparse OSM brand tagging in India rather than KML inaccuracy, but a manual sample audit on Google Maps would lift this from a coverage statistic to an accuracy figure.

A fourth, lesser, limitation is that MILP scaling to $N \geq 100$ is impractical with CBC; a commercial solver (Gurobi/CPLEX) would close some of that gap. The heuristics already produce competitive bounds, and the `auto` backend in `src/savrptw/solvers/milp.` switches to commercial solvers when their Python bindings are present.

7.3 Threats to Validity

The customer placement model uses residential-density weighting on the OSM `landuse` layer rather than a population-density raster. Where census or WorldPop microdata is available, plugging it into `instance.generator` would tighten the demand model. The behavioural compliance prior is anchored on a single field-data citation (Das et al. 2020), not a multi-study meta-analysis; the Beta-prior sensitivity sweep in Chapter 6 is the safeguard against mis-specification.

7.4 Empirical Findings

The completed experiment grid (428 feasible rows across five cities and four problem sizes) yields three concrete empirical conclusions.

GA dominates ALNS at large N. At $N = 200$, the giant-tour GA with Bellman–Ford split produces lower mean F_1 than ALNS in every city with more than one seed (e.g. Bengaluru: GA $F_1 = 4.89$ vs. ALNS $F_1 = 6.94$; Delhi: GA 5.82 vs. ALNS 9.66), while running approximately 325× faster (mean 26 s vs. 8,407 s). The Bellman–Ford decoder pre-solves the route-assignment sub-problem at each fitness evaluation, making the GA remarkably effective for this formulation relative to the destroy-repair operators of ALNS.

Both solvers scale near-linearly in F_1 with N. Doubling N from 50 to 100 approximately doubles mean F_1 (Bengaluru GA: $1.74 \rightarrow 3.35$; ALNS: $1.64 \rightarrow 3.98$), consistent with the soft-time-window penalty accumulating proportionally to the number of customers served late.

City geometry matters. Gurugram and Mumbai produce elevated F_1 at $N \geq 50$ relative to their Blinkit dark-store counts, driven by fewer depot clusters relative to demand spread. Bengaluru provides the most stable per-seed results and is the recommended primary city for sensitivity studies.

7.5 Summary of Contributions

The contributions of this thesis are as follows.

1. A Safety-Aware VRPTW formulation that treats per-route crash exposure as a hard ε-constraint rather than as an objective component, with a separate fleet-wide residential budget ($\bar{H}$) and a per-route residential cap ($\bar{H}_k = 100$).

2. A documented public-source crash-risk calibration chain (BASM) anchored on Ministry of Road Transport and Highways (2023) aggregates, Mohan et al. (2017) ordering constraints, and an OpenStreetMap structural proxy; the explicit π_{ij} formula in Chapter 4 means the proxy is reproducible from the manuscript alone.

3. A unified solver/evaluation contract under which a PuLP-based MILP, a giant-tour GA with Bellman–Ford optimal split (Prins 2004), and an Adaptive Large Neighbourhood Search (Ropke & Pisinger 2006) all evaluate F_1 through identical code, eliminating the apples-to-pears bias common in VRP solver comparisons.

4. An empirical finding that the GA with Bellman–Ford split outperforms ALNS at $N \geq 100$ in both solution quality and wall-clock time on the SA-VRPTW formulation, with a 325× speed advantage at $N = 200$ across five Indian cities.

5. A five-city empirical study on real Blinkit dark-store coordinates from a publicly committed KML snapshot, with Hyderabad explicitly excluded because the snapshot has zero in-bbox placemarks.

6. A reproducibility surface that includes Hydra configuration, seeded experiment runs, a manifest-checkpointed grid driver, and a calibration note that prevents production runs from silently using uncalibrated risk constants.

7.6 Conclusion

This thesis presented an SA-VRPTW formulation for q-commerce last-mile delivery in Indian cities that handles crash safety as an explicit ε-constraint, calibrates per-edge crash probability through a documented public-source chain, and is implemented under a unified solver and evaluation contract across MILP, GA, and ALNS. The empirical study on 428 feasible instances across five cities demonstrates that the giant-tour GA with Bellman–Ford split is both faster and higher-quality than ALNS at $N \geq 100$, with a 325× wall-clock advantage at $N = 200$. The full pipeline is publicly released under the MIT licence.

7.7 Future Work

The next phase is operational and empirical rather than conceptual.

Behavioural meta-analysis. Replace the single-study Das et al. (2020) compliance prior with a multi-study meta-analysis covering Indian gig-rider populations beyond Bengaluru.

Real-time congestion. Replace the static BPR model with a real-time congestion feed where available; the BPR model in `conf/congestion/bpr.yaml` is already isolated, so the swap is local.

Stochastic demand. Extend the formulation to a stochastic demand variant in which order arrivals across the dispatch window are modelled by a non-stationary point process rather than a uniform $[0, 5]$ minute draw.

Risk-aware path selection. Lift the super-graph restriction so that the optimisation jointly selects customer orderings and physical street paths. This is a materially harder model, but it is the natural next step beyond the current safety-aware-*ordering* formulation.

Direct crash microdata. If access to city-level crash microdata or validated black-spot inventories becomes possible, BASM can be upgraded from a public-aggregate surrogate to a more direct empirical edge-risk model without changing the surrounding solver architecture.

Bibliography

Boeing, G. (2017), 'OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks', *Computers, Environment and Urban Systems* **65**, 126–139.

Bureau of Public Roads (1964), Traffic assignment manual, Technical report, U.S. Department of Commerce, Urban Planning Division, Washington, DC, USA.

Das, S., Mukherjee, A. & Banerjee, R. (2020), 'Helmet compliance among gig-economy delivery riders in indian cities', *Accident Analysis & Prevention* **141**, 105517.

Desrochers, M., Desrosiers, J. & Solomon, M. M. (1992), 'A new optimization algorithm for the vehicle routing problem with time windows', *Operations Research* **40**(2), 342–354.

Ehrgott, M. (2005), *Multicriteria Optimization*, 2nd edn, Springer, Berlin, Germany.

Erkut, E. & Verter, V. (1998), 'Modeling of transport risk for hazardous materials', *Operations Research* **46**(5), 625–642.

Fu, Z., Eglese, R. & Li, L. Y. O. (2005), 'A new tabu search heuristic for the open vehicle routing problem', *Journal of the Operational Research Society* **56**(3), 267–274.

Holland, J. H. (1975), *Adaptation in Natural and Artificial Systems*, University of Michigan Press, Ann Arbor, MI, USA.

Hoseinzadeh, M., Mohammadi, M. & Mirjalili, S. (2020), 'A safety-aware vehicle routing problem with cumulative risk', *Computers & Industrial Engineering* **149**, 106786.

Koskosidis, Y. A., Powell, W. B. & Solomon, M. M. (1992), 'An optimization-based heuristic for vehicle routing and scheduling with soft time window constraints', *Transportation Science* **26**(2), 69–85.

Miller, C. E., Tucker, A. W. & Zemlin, R. A. (1960), 'Integer programming formulation of traveling salesman problems', *Journal of the ACM* **7**(4), 326–329.

Ministry of Road Transport and Highways (2023), Road accidents in India 2022, Technical report, Government of India, New Delhi, India. Available from the Ministry of Road Transport and Highways.

Mohan, D., Bangdiwala, S. I. & Villaveces, A. (2017), 'Urban street structure and traffic safety', *Journal of Safety Research* **62**, 63–71.

Prins, C. (2004), 'A simple and effective evolutionary algorithm for the vehicle routing problem', *Computers & Operations Research* **31**(12), 1985–2002.

Ropke, S. & Pisinger, D. (2006), 'An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows', *Transportation Science* **40**(4), 455–472.

Salmon, J., Martínez, A. & Verma, R. (2023), 'Multi-objective ε-constraint methods for safety-aware logistics', *European Journal of Operational Research* **308**(3), 1108–1124.

Sinchaisri, W. P., Lobel, I. & Allon, G. (2023), 'Operating under SLA pressure: Evidence from quick-commerce dispatch', *Manufacturing & Service Operations Management* . to be published.

Solomon, M. M. (1987), 'Algorithms for the vehicle routing and scheduling problems with time window constraints', *Operations Research* **35**(2), 254–265.

Taillard, E. D., Badeau, P., Gendreau, M., Guertin, F. & Potvin, J.-Y. (1997), 'A tabu search heuristic for the vehicle routing problem with soft time windows', *Transportation Science* **31**(2), 170–186.

Toth, P. & Vigo, D., eds (2002), *The Vehicle Routing Problem*, SIAM, Philadelphia, PA, USA.

Transportation Research Board (2010), *Highway Capacity Manual (HCM) 2010*, National Academies of Sciences, Engineering, and Medicine, Washington, DC, USA.

Wilson, E. B. (1927), 'Probable inference, the law of succession, and statistical inference', *Journal of the American Statistical Association* **22**(158), 209–212.

Zografos, K. G. & Androutsopoulos, K. N. (2004), 'A heuristic algorithm for solving hazardous materials distribution problems', *European Journal of Operational Research* **152**(2), 507–519.